Features and deployment

14B. Features & Deployment

Questions: 4 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Microsoft

Q 14.5 How does Next.js Middleware work? What are common use cases?

Difficulty: Senior **Asked at:** Razorpay, Flipkart, Amazon

Middleware runs before a request reaches the page or API route. It can redirect, rewrite, modify headers, or return a response directly. It executes at the edge (CDN), not on your origin server, which makes it fast for geographically distributed users.

TYPESCRIPT Copy

```typescript
// middleware.ts (root of the project — runs on every request)
import { NextRequest, NextResponse } from 'next/server';

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // 1. Authentication check
  const token = request.cookies.get('session')?.value;
  if (pathname.startsWith('/dashboard') && !token) {
    return NextResponse.redirect(new URL('/login', request.url));
  }
```

```
  // 2. Geolocation-based redirect
  const country = request.geo?.country;
  if (pathname === '/' && country === 'IN') {
    return NextResponse.rewrite(new URL('/in', request.url));
  }

  // 3. Add security headers
  const response = NextResponse.next();
  response.headers.set('X-Frame-Options', 'DENY');
  response.headers.set('X-Content-Type-Options', 'nosniff');

  // 4. A/B testing - route to variant
  const bucket = request.cookies.get('ab-bucket')?.value || (Math.random()
  if (pathname === '/pricing') {
    response.cookies.set('ab-bucket', bucket);
    return NextResponse.rewrite(new URL(`/pricing/${bucket}`, request.url)
  }

  return response;
}

// Only run middleware on specific paths (don't run on static assets)
export const config = {
  matcher: ['/((?!_next/static|_next/image|favicon.ico).*)'],
};
```

Common use cases:

- **Authentication/Authorization:** Redirect unauthenticated users before the page even starts rendering. Faster than checking auth inside the page component.
- **Geo-routing:** Redirect to country-specific versions (`/in`, `/us`) based on IP geolocation.
- **A/B testing:** Assign users to test buckets and rewrite to variant pages without the user seeing a different URL.
- **Bot detection:** Block scrapers or rate-limit suspicious traffic before it hits your application.
- **Maintenance mode:** Return a maintenance page for all routes with one middleware check.

- **Custom headers:** Add security headers, CORS headers, or caching headers to all responses.

Limitations: Middleware runs on the Edge Runtime, which is a subset of Node.js. No `fs`, no native Node modules, limited library support. Keep middleware lightweight — complex logic belongs in API routes or server components.

Q 14.6 How does Next.js optimize images and fonts?

Difficulty: Mid **Asked at:** Flipkart, Amazon, Swiggy

`next/image` **— automatic image optimization:**

TSX Copy

```tsx
import Image from 'next/image';

function ProductCard({ product }) {
  return (
    <Image
      src={product.imageUrl}
      alt={product.name}
      width={400}
      height={300}
      placeholder="blur"
      blurDataURL={product.blurHash}
      sizes="(max-width: 768px) 100vw, 400px"
      priority={false}  // Set true for above-the-fold / LCP images
    />
  );
}
```

What Next.js does automatically:

- **Format conversion:** Serves WebP or AVIF based on browser support — no manual conversion needed.
- **Responsive sizing:** Generates multiple sizes at build time or on-demand. The `sizes` prop tells the browser which size to request.

- **Blur placeholder:** Shows a blurred low-resolution version while the full image loads — eliminates CLS.
- **Caching:** Optimized images are cached at the CDN. Subsequent requests serve the cached version.
- **No CLS:** Width and height are required — the browser reserves exact space before the image loads.

`next/font` — zero-layout-shift font loading:

TSX Copy

```tsx
import { Inter, JetBrains_Mono } from 'next/font/google';

const inter = Inter({
  subsets: ['latin'],
  display: 'swap',
  variable: '--font-sans',
});

const jetbrainsMono = JetBrains_Mono({
  subsets: ['latin'],
  variable: '--font-mono',
});

export default function RootLayout({ children }) {
  return (
    <html className={`${inter.variable} ${jetbrainsMono.variable}`}>
      <body>{children}</body>
    </html>
  );
}
```

What `next/font` does automatically:

- **Self-hosts Google Fonts:** Downloads font files at build time and serves from your domain — no request to `fonts.googleapis.com`.

- **Zero CLS:** Automatically generates `size-adjust`, `ascent-override`, and `descent-override` CSS for the fallback font, matching it to the custom font's metrics. When the custom font loads, there's no visible shift.
- **Subset optimization:** Only includes the character sets you specify (latin, cyrillic, etc.).
- **Preloading:** Adds `<link rel="preload">` for font files automatically.

Follow-up: "Can you use `next/image` with external image sources?"

Yes, but you must whitelist domains in `next.config.js`:

JAVASCRIPT Copy

```
module.exports = {
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: 'images.cdn.example.com' },
      { protocol: 'https', hostname: '**.cloudinary.com' },
    ],
  },
};
```

Q 14.7 What are the deployment options for Next.js? Edge vs Serverless vs Node.js runtime.

Difficulty: Senior **Asked at:** Razorpay, Amazon, Flipkart

Vercel — the default:

Built by the Next.js team. Zero-config deployment. Automatic preview deployments per PR. Edge middleware, serverless functions, and ISR work out of the box. The simplest option, but vendor lock-in is the trade-off.

Self-hosted Node.js server:

BASH Copy

```
next start  # Starts a Node.js HTTP server
```

Full Node.js runtime — all features work (middleware, API routes, ISR, image optimization). Deploy behind nginx or a load balancer. You manage the infrastructure (scaling, monitoring, SSL).

When to use: your organization can't use third-party hosting (compliance, data residency), you need full control over the server, or you're running alongside other backend services on the same infrastructure.

Docker:

DOCKERFILE Copy

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/.next ./.next
COPY --from=builder /app/public ./public
COPY --from=builder /app/package*.json ./
RUN npm ci --production
EXPOSE 3000
CMD ["npm", "start"]
```

Deploy to AWS ECS, Google Cloud Run, or any container orchestration platform.

Static export — no server needed:

JAVASCRIPT Copy

```
  output: 'export',  // Generates static HTML for all pages
};
```

Produces a `out/` folder with pure HTML/CSS/JS files. Deploy to any static host (S3 + CloudFront, Netlify, GitHub Pages). Limitations: no SSR, no API routes, no ISR, no middleware, no image optimization.

When to use: documentation sites, marketing pages, any site that doesn't need server-side features.

Runtime comparison:

JAVASCRIPT Copy

```
Runtime    | Features                  | Cold Start | Best For
-----------+---------------------------+------------+----------------
Node.js    | Everything works          | None       | Full control,
           |                           | (always on)| complex apps
Serverless | Most features, auto-      | 200-500ms  | Variable
           | scaling, pay-per-use      |            | traffic apps
Edge       | Middleware, simple        | <50ms      | Auth checks,
           | responses (limited API)   |            | redirects, headers
Static     | No server features        | None       | Docs, marketing,
           |                           | (CDN-only) | blogs
```

Follow-up: "What's the cost difference between Vercel and self-hosting?"

Vercel's free tier covers small projects. Paid plans start at $20/month per team member. At scale (millions of function invocations, hundreds of GB of bandwidth), Vercel can be expensive. Self-hosting on a VPS ($20-100/month) is cheaper for consistent high traffic but requires DevOps effort. The decision is cost vs convenience.

Q 14.8 When would you choose Next.js vs Remix vs Astro?

Difficulty: Senior Asked at: Google, Razorpay, Flipkart

Next.js — the full-stack React framework:

Best for: applications that need SSR, SSG, ISR, and API routes. E-commerce, SaaS dashboards, content platforms. The largest ecosystem, most job postings, and strongest Vercel integration.

Weaknesses: complex caching behavior (Next.js 15 changed fetch caching defaults), App Router learning curve, Vercel-optimized features may not work as well self-hosted.

Remix — web standards first:

Best for: applications with complex forms and mutations, progressive enhancement requirements, teams that prefer web standards (FormData, Request/Response) over framework abstractions. Remix's data loading model (loaders + actions) is simpler than Next.js's fetch caching.

Weaknesses: smaller ecosystem, no built-in ISR/SSG (Remix is SSR-only by design — all rendering happens at request time), fewer deployment integrations.

Astro — content-first, minimal JS:

Best for: content-heavy sites (blogs, documentation, marketing) where 90%+ of the page is static. Ships zero JavaScript by default — you opt in to interactivity per-component with client directives. Supports React, Vue, Svelte, and Solid components in the same project.

Weaknesses: not suitable for highly interactive applications (dashboards, real-time apps, complex forms). Islands Architecture doesn't work well when most of the page needs interactivity.

The decision:

JAVASCRIPT Copy

```
"I'm building a ..."                    | Use
  ──────────────────────────────────────┼──────────
  SaaS product / dashboard              | Next.js
  E-commerce site                       | Next.js
  Content site / blog / docs            | Astro
  Form-heavy app with mutations         | Remix
```

Marketing site with minimal JS	Astro
[illegible]	Next.js or Remix
Multi-framework component demo	Astro

What interviewers want to hear: Not "Next.js is best." They want you to reason about the trade-offs for a specific use case. "For this documentation site, I'd use Astro because 95% of the content is static markdown and we'd ship almost zero JavaScript. But for the companion SaaS dashboard, I'd use Next.js because it needs SSR for dynamic user data, API routes for backend logic, and ISR for cached but frequently updated content."

Follow-up: "What about SvelteKit?"

SvelteKit is the Svelte equivalent of Next.js — SSR, SSG, file-based routing, server actions. If the team prefers Svelte over React (smaller bundle, no virtual DOM, less boilerplate), SvelteKit is an excellent choice. The limitation is ecosystem — React has 10x the component library and tooling availability.

QUICK CHECK — no marks, just for you.

1. Why does next/image require you to pass width and height?

- ○ So the browser can reserve the exact space before the image loads, avoiding layout shift
- ○ So the image is excluded from lazy loading and fetched eagerly
- ○ So Next.js knows which image format (WebP or AVIF) the browser supports
- ○ So the optimized image can be cached at the CDN instead of the origin

Check answer

2. You set output: 'export' in next.config.js to deploy a Next.js site to S3 + CloudFront. Which of the following will still work?

- Middleware, because it runs at the CDN edge rather than on the origin server
- Nothing server-side — the build produces static HTML/CSS/JS only, with no SSR, API routes, ISR, middleware, or image optimization
- API routes, because they are bundled into the static output as client-side handlers

Check answer

3. Your middleware.ts needs to read a config file from disk with the Node fs module before deciding whether to redirect. According to the lesson, what is the problem and where should this logic go?

- Middleware runs on the origin server, so fs works but the file read blocks the CDN cache and should move to a serverless function
- Middleware can use fs only when the app is deployed to Vercel; self-hosted Node deployments must move it to a client component
- Middleware runs on the Edge Runtime, a subset of Node.js with no fs and no native modules, so heavy logic belongs in API routes or server components
- Middleware can use fs, but only inside the config matcher export, so the read must be moved there

Check answer

Frontend System Desi...

LEARN MORE

ARTICLES & DOCS

Next.js documentation
nextjs.org

VIDEOS



Server Actions: Next.js's best feature
YouTube



Server Actions in 5 minutes
YouTube



Server Actions tutorial
YouTube

Hand-picked references for this lesson.